6. To reduce human effort and promote sustainable waste management.

3.1.3 Functional Requirements

The system functionalities are divided into backend, frontend, and AI model requirements.

A. Backend (Fastapi) Requirements

- Accept an uploaded image from the user.
- It preprocess the image by resizing it and normalizing it
- Load the trained MobileNetV2 `.h5` waste classification model.
- Predict the waste category and return confidence score.
- Provide JSON response to the frontend.
- It will also handle errors such as invalid file types and missing inputs.

B. Frontend (Reactjs) Requirements

- User can upload an image or can even use a live camera feed for ease.
- It will also display the preview of the image.
- Send the image to the backend API.
- Display predicted category and confidence percentage.
- It will also show additional info like:
    1. Carbon footprint
    2. Disposal method
    3. Recycling and Reuse suggestions

C. Model Requirements

- Train Deep Learning model on 10-class dataset which were like 19k images approximately.
- Achieve at least 90%+ accuracy on validation data.
- Optimize model using fine-tuning and augmentation.

- Save trained model as `mobilenetv2_model.h5`.
- Perform evaluation using:
  1. Confusion Matrix
  2. Classification report
  3. Accuracy curves and loss graphs

3.1.4 Non-Functional Requirements

A. Performance Requirements

- Model prediction should be completed within 1–2 seconds.
- Frontend must remain responsive at all times.
- Backend must handle multiple user requests simultaneously.

B. Usability Requirements

- Simple UI for non-technical users.
- Smooth, intuitive experience on desktop and mobile.
- Clear instructions for image upload and detection.

C. Reliability Requirements

- Backend must run without unexpected crashes.
- Model should consistently give accurate outputs (94% validation accuracy).

D. Security Requirements

- Only image files should be accepted.
- Backend should prevent harmful file uploads.

- CORS enabled for secure communication between frontend and backend.

E. Scalability Requirements

- Additional waste categories can be added to the dataset.
- Real-time video detection support.
- Integration with IoT bins in future versions.

3.1.5 System Constraints

- Hardware limitations of end-user devices may affect performance.
- Camera quality impacts real-time detection accuracy.
- Internet connectivity is required for accessing backend API.
- Dataset imbalance for certain classes influences precision.

3.1.6 Feasibility Analysis

Technical Feasibility

- MobileNetV2 is lightweight and efficient on CPUs.
- FastAPI provides high-performance backend support.
- ReactJS allows flexible UI development.

Operational Feasibility

- Can be used in households, colleges, municipalities, and industries.
- Easy to operate with minimal training.

Economic Feasibility

- Open-source tools reduce overall cost.
- No paid libraries or cloud services required.

3.2 Project Design and Architecture

3.2.1 System Architecture

The overall architecture of our model consists of three major components which all come into play at the same time:

1. Frontend (ReactJS)

- Provides UI for users.
- Allows image upload or real-time camera capture.
- Sends the image to the FastAPI backend.
- Displays prediction, carbon footprint, disposal method, and recycling suggestions.

2. Backend (FastAPI)

- Loads the trained MobileNetV2 `.h5` model.
- Receives image input from frontend.
- Performs preprocessing and classification.
- Returns the predicted class and confidence score.

3. Deep Learning Model (MobileNetV2)

- Lightweight CNN pre-trained on ImageNet.
- Fine-tuned on 10 waste categories (~19,000 images).
- Outputs probabilities for each waste type.

3.2.2 Architecture Diagram

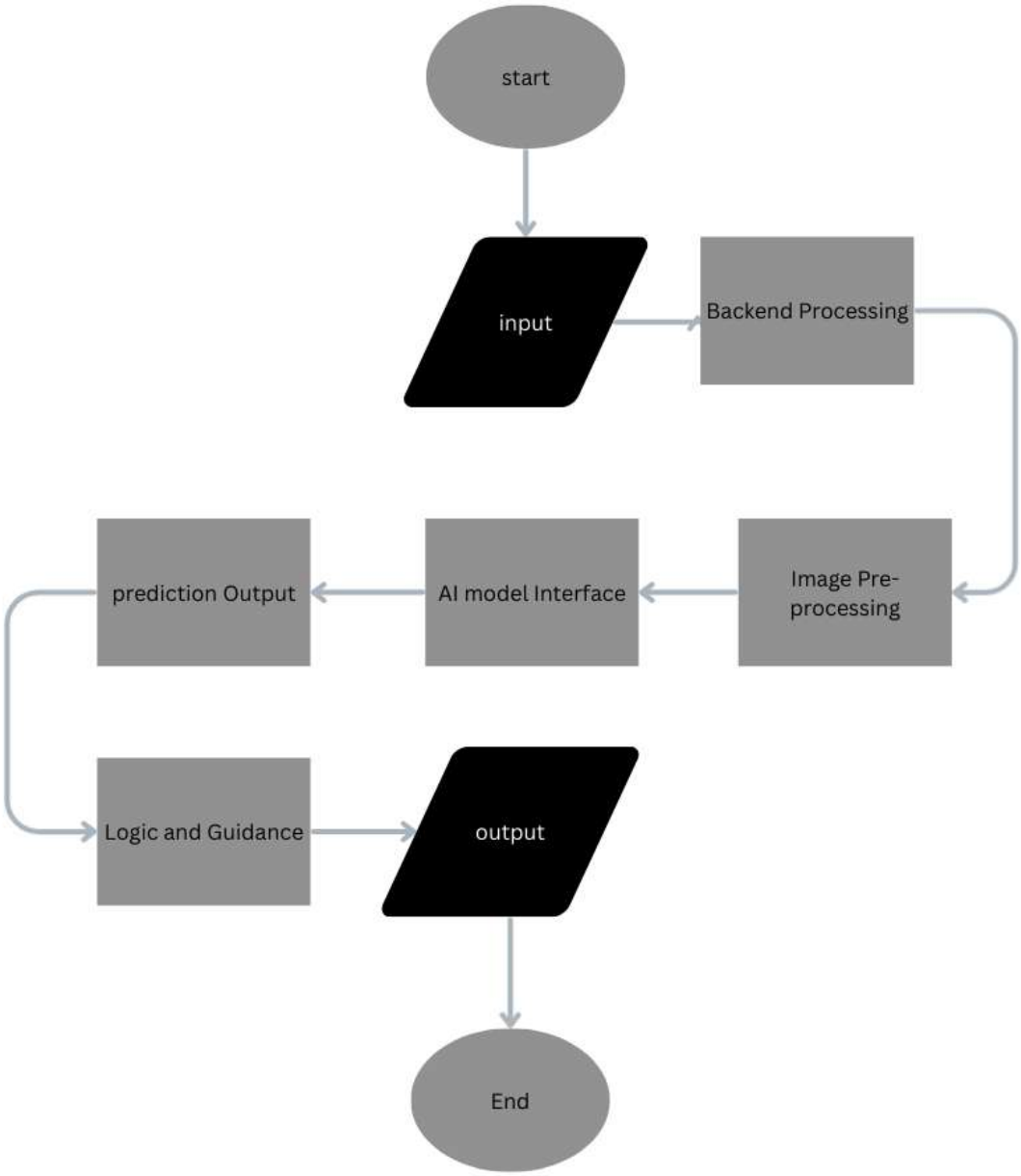

Figure 3.1 : Project Design and Architecture

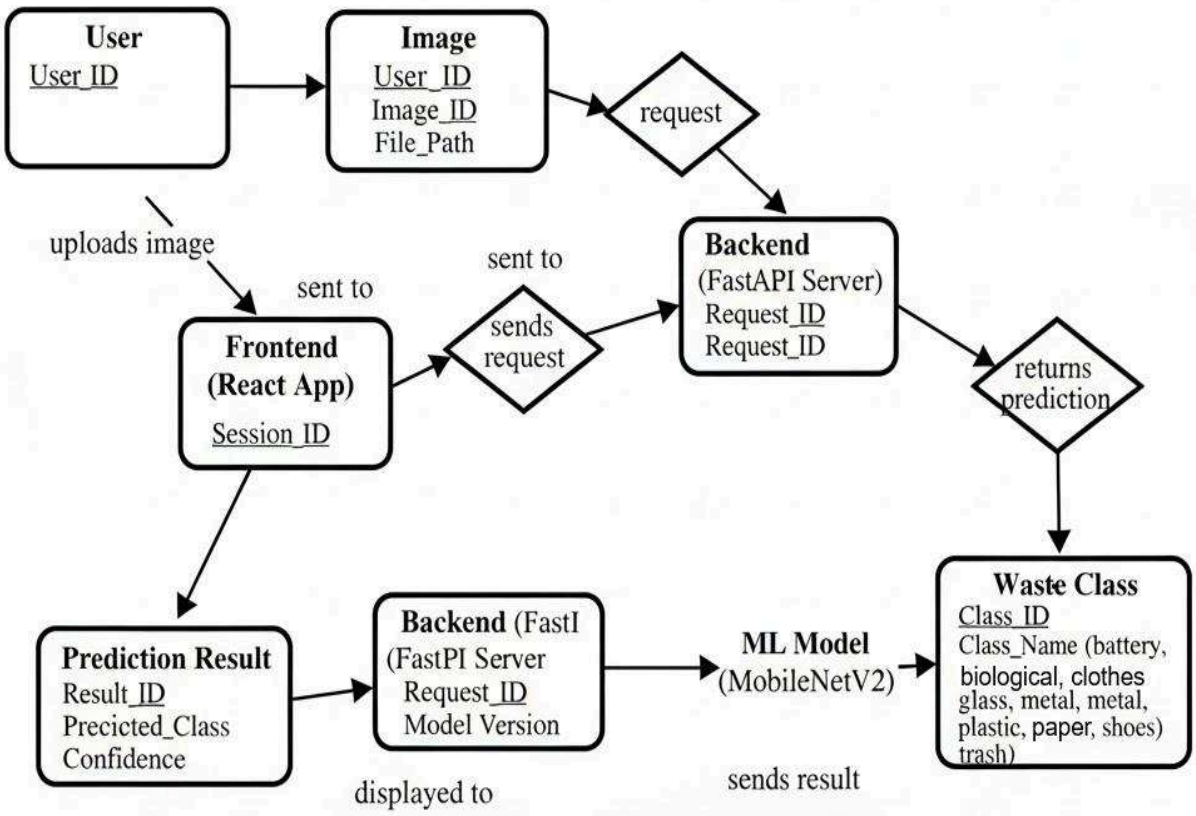

Figure 3.2 : Workflow Diagram

3.2.3 Use-Case Diagram

Primary Actor: User (Web User)

Use Cases:

- Upload an image
- Capture real-time image from camera
- View prediction + confidence
- Get waste management suggestions
- Understand carbon footprint impact

3.3 Data Preparation

The dataset used contains 19,762 labeled waste images across 10 categories:

battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes, trash

3.3.1 Data Sources

- Combined Waste Classification dataset from open repositories (TrashNet + Extended dataset).
- Images downloaded from research collections and public datasets.

3.3.2 Preprocessing Steps

1. Directory Structure

```
dataset/
  ├── battery/
  ├── biological/
  ├── cardboard/
  ├── clothes/
  ├── glass/
  ├── metal/
  ├── paper/
  ├── plastic/
  ├── shoes/
  └── trash/
```

2. Augmentation Applied

- Rotation (20°)
- Random zoom
- Width & height shifts
- Shearing
- Horizontal flips
- Rescaling (1/255)

3.3.3 Splitting

- Training set: 90%
- Validation set: 10%

Done using:

```
ImageDataGenerator(validation_split=0.1)
```

3.3.4 Image Preprocessing

Each image is:

- Converted to RGB
- Resized to 224×224
- Normalized to [0,1]
- Expanded to batch dimension

3.4 Implementation

3.4.1 Technology Stack

Component	**Technology**
Deep Learning	TensorFlow, Keras, MobileNetV2
Backend	FastAPI
Frontend	ReactJS
Styling	CSS
Streaming	Webcam JS / MediaDevices API
Evaluation	Scikit-learn

3.4.2 Model Implementation (MobileNetV2)

Training Code Snippet

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Figure 3.3 - Screenshot of Training Code

Initial Training

```

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(train_ds, validation_data=val_ds, epochs=10)
```

Figure 3.4 - Screenshot of Initial Training

Fine-tuning

```
base_model.trainable = True
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_ft = model.fit(train_ds, validation_data=val_ds, epochs=10)
```

Figure 3.5 - Screenshot of Fine- tuning

Saving Model

```

model.save("../backend/model/mobilenetv2_model.h5")
print("\nModel saved successfully! ")
```

Figure 3.6 - Screenshot of Saving Model

3.4.3 Backend (FastAPI)

Prediction Endpoint

```
# --------- PREDICTION ROUTE ---------
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    if model is None:
        return {"error": "Model not loaded on server."}

    try:
        img = Image.open(file.file).convert("RGB")
        img = img.resize((224, 224))
        img = np.array(img) / 255.0
        img = np.expand_dims(img, axis=0)

        prediction = model.predict(img)
        index = int(np.argmax(prediction))
        confidence = float(np.max(prediction))

        return {
            "class": classes[index],
            "confidence": round(confidence, 4)
        }

    except Exception as e:
        return {"error": f"Failed to process image: {e}"}

# --------- RUN SERVER ---------
```

Figure 3.7 - Screenshot of FastApi Prediction endpoint implementation

3.4.4 Frontend (ReactJS)

Uploading Image & Detecting Waste

```
    setErrorMsg("");

    const formData = new FormData();
    formData.append("file", image);

    try {
      const response = await axios.post("http://127.0.0.1:8000/predict", formData, {
        headers: { "Content-Type": "multipart/form-data" },
      });

      let cls = response.data.class?.toString().trim().toLowerCase();
      setPrediction(cls);
      setConfidence(response.data.confidence);
    } catch (err) {
      setErrorMsg("Prediction Error");
    }

    setLoading(false);
  };
```

Figure 3.8 - Screenshot of ReactJS Image Upload and waste Detection Function

3.4.5 Real-Time Camera Detection

Using `navigator.mediaDevices.getUserMedia()` API.

3.4.6 Backend Connectivity

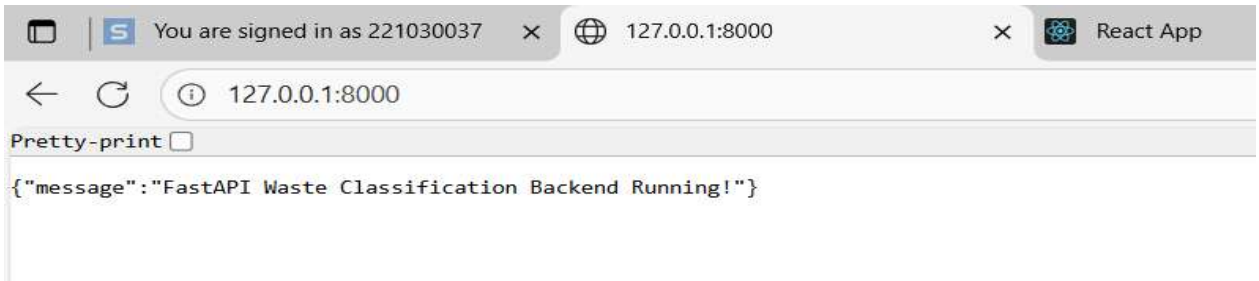

Figure 3.9 - Screenshot of Backend Connectivity

3.4.7 Frontend UI Working

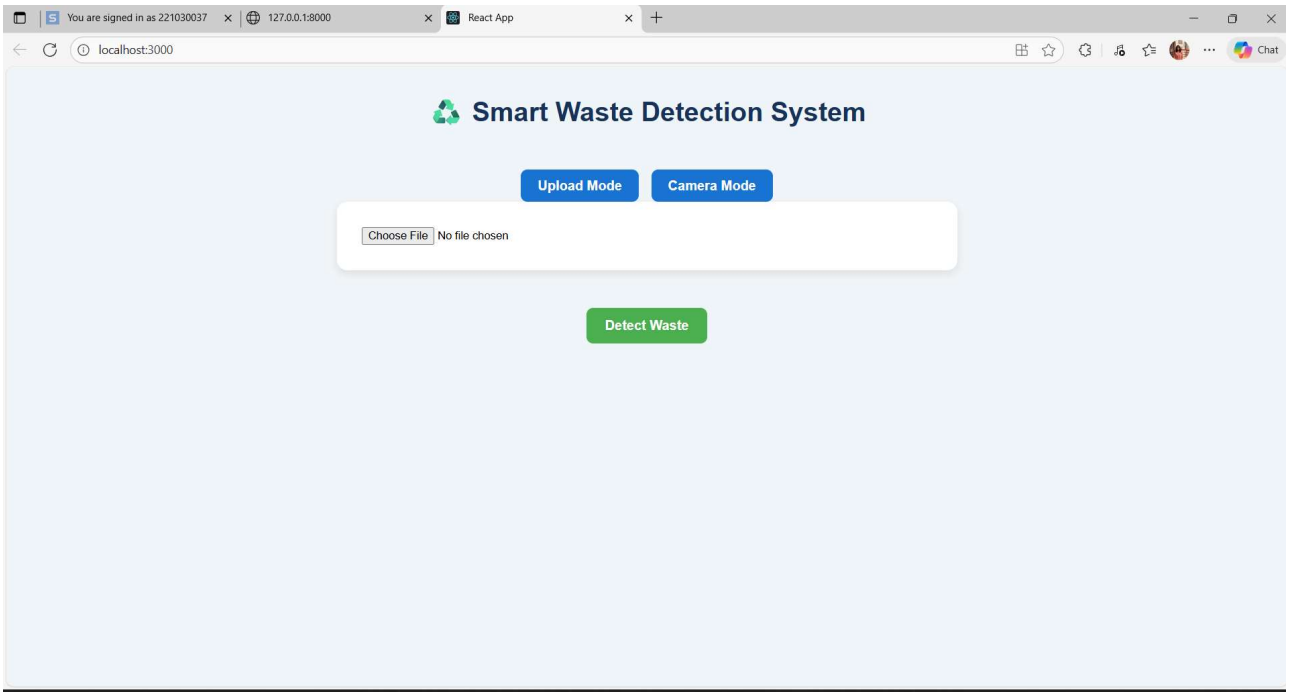

Figure 3.10 - Screenshot of the Home Page

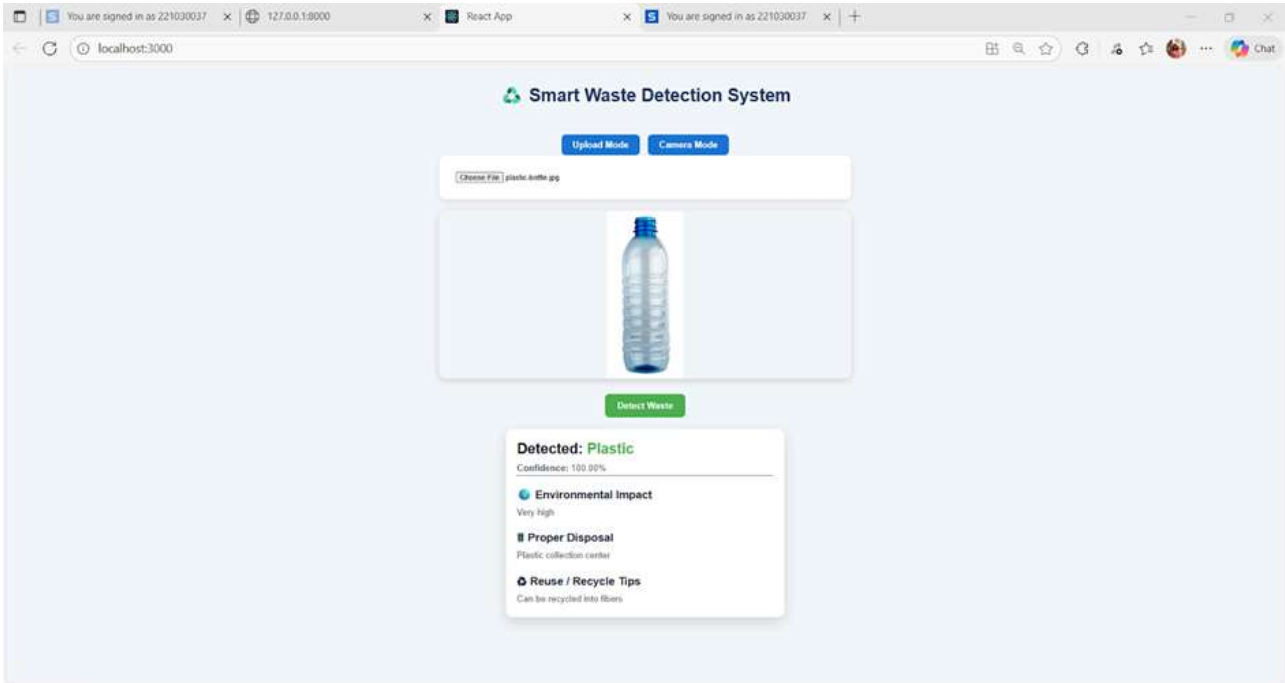

Figure 3.11 - Screenshot of Waste Detection Result Page

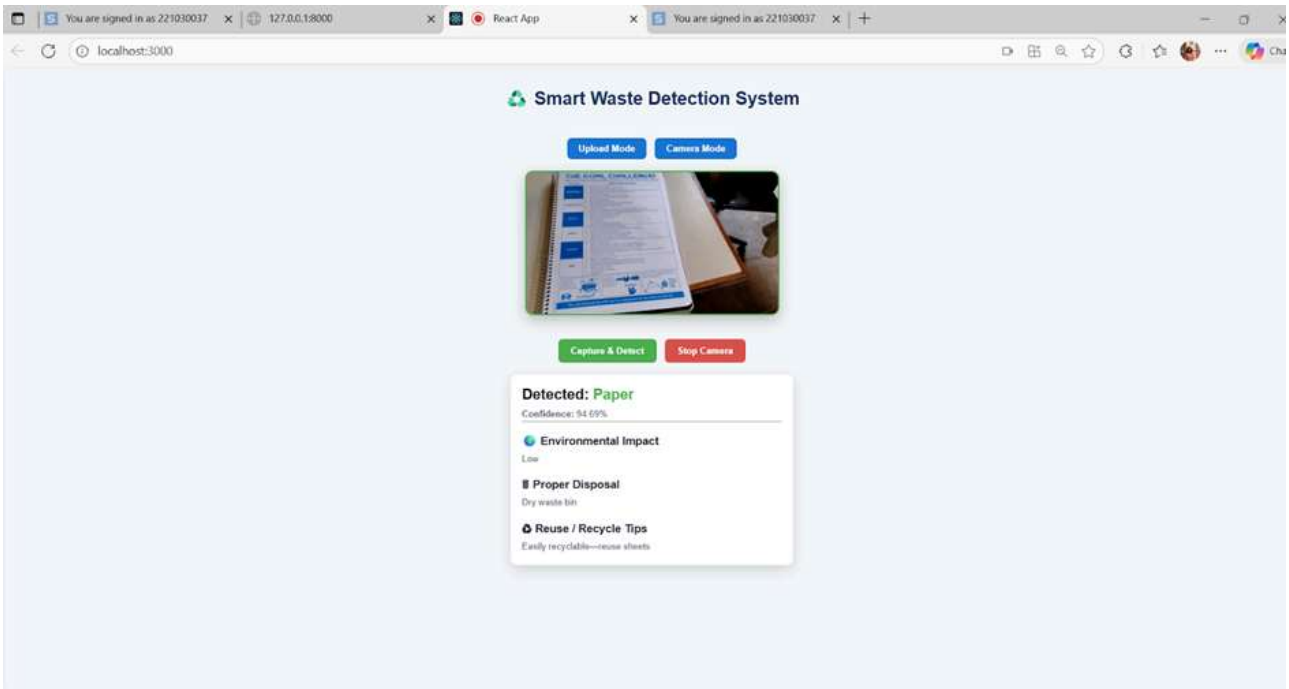

Figure 3.12 - Screenshot of Live camera Waste Detection Page

3.5 Key Challenges and Solutions

1. Large Dataset Handling

The real challenge was training on 19k images which caused slow training.
The solution was

- We took help of MobileNetV2 instead of heavy models which was a good choice.
- We applied batch loading and did augmentation which decreased training time.

2. Missing Dependencies / TensorFlow Errors

Challenge: TensorFlow, PIL and SciPy errors.
Solution:

- We installed required libraries like Pillow, scipy and tensorflow

3. Overfitting

Challenge: our first initially drafted model was overfitted due to high similarity in training images.
Solution:

- We added augmentation which helped
- We used dropout
- We also finetuned only the top layers instead of all

4. Backend Loading Time

Challenge: Model loading took a few seconds on startup.
Solution:

- We loaded model only once globally which decreased loadup time
- We also enabled and used async FastAPI routes

5. Connecting Frontend & Backend

Challenge: API requests were found to be blocked due to CORS errors.
Solution:

```
allow_origins=["*"]
```

6. Real-Time Camera Classification

Challenge: Processing video frames in real-time requires optimization which was challenging.
Solution:

- We reduced the frequency of frame capturing
- We converted the preview to 224×224 px before sending it

7. Designing User-Friendly Interface

Challenge: Displaying predictions meaningfully.
Solution:

- Added environmental impact
- Reuse/recycle suggestion